AI-Chatbot-Mizushima

Servidor MCP, agente y skills · Documentación

https://mizushima-mcp.onrender.com/mcp
github.com/heroleni/AI-Chatbot-Mizushima

Contenido

Visión general

Documentación del código

Documentación del agente (Mizushima)

Documentación de las skills

Referencia del servidor MCP

Conectarse desde cualquier cliente

Despliegue

Anexo A. Skill para agentes (SKILL.md)

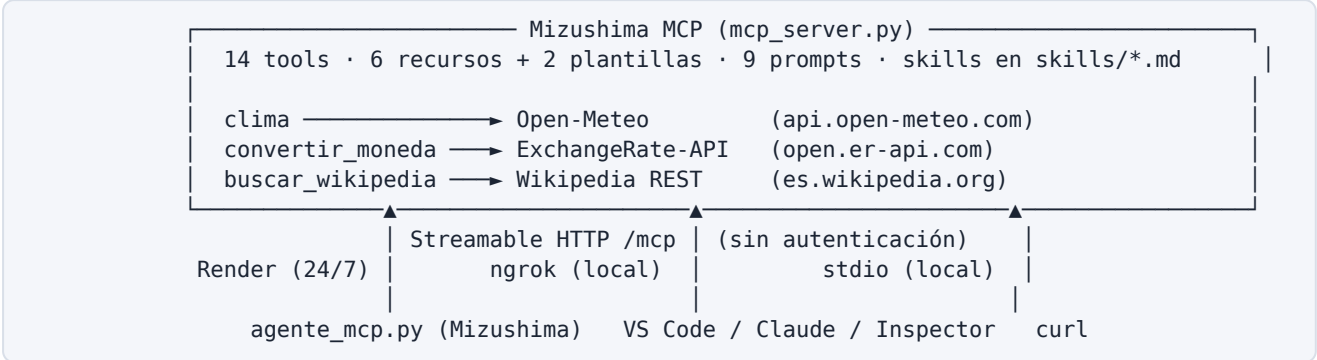
Anexo B. Skills del servidor

Visión general

Servidor **MCP** (Model Context Protocol) con tools, recursos, prompts y skills, desplegado en internet **sin autenticación** para que cualquier cliente MCP lo use. Incluye a **Mizushima**, un agente por terminal (OpenAI o Gemini) que se conecta a ese servidor desde cualquier computador.

Servidor MCP	**https://mizushima-mcp.onrender.com/mcp** (Streamable HTTP, sin autenticación)
Información del servidor	https://mizushima-mcp.onrender.com
Repositorio	https://github.com/heroleni/AI-Chatbot-Mizushima

> Está en el plan gratis de Render: si lleva más de 15 minutos sin uso, la primera petición tarda unos 50 segundos.



Criterios de aceptación

Criterio	Cómo se cumple
Al menos 3 recursos	6 recursos (`info://servidor`, `notas://lista`, `notas://todas`, `tareas://pendientes`, `tareas://todas`, `skills://indice`) y 2 plantillas (`notas://{titulo}`, `skills://{nombre}`)
Al menos 5 herramientas	14 herramientas
2 consumen APIs de internet de servicios distintos	3: `clima` (Open-Meteo), `convertir_moneda` (ExchangeRate-API) y `buscar_wikipedia` (Wikipedia)
Al menos 2 prompts para las APIs	3: `reporte_clima`, `presupuesto_viaje` (usa 2 APIs) e `investigar_tema`
Desplegado para cualquier cliente	Render (siempre activo) y ngrok (local). Streamable HTTP con respuestas JSON: funciona hasta con `curl`
Sin autenticación	El servidor no pide token, clave ni usuario
Skill para agentes	`agent-skills/mizushima-mcp/SKILL.md`

Criterio	Cómo se cumple
Documentación y código	Este README y la carpeta `docs/`: código, agente y skills. 16 pruebas en `tests/`

Inicio rápido

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python -m pip install -r requirements.txt
```

Quiero…	Comando
Correr las pruebas	`python -m pytest -v`
Usar el agente con el servidor local	`python agente_mcp.py`
Usar el agente con el servidor desplegado	`python agente_mcp.py --url https://mizushima-mcp.onrender.com`
Levantar el servidor HTTP local	`MCP_TRANSPORT=streamable-http python mcp_server.py`
Verificar un despliegue	`python verificar_despliegue.py https://mizushima-mcp.onrender.com`

Para el agente, copia `.env.example` como `.env` y pon tu `OPENAI_API_KEY` y/o `GOOGLE_API_KEY`. **El servidor no necesita ninguna clave.**

> En Windows 11, si el Control inteligente de aplicaciones bloquea librerías (`DLL load failed… Control de aplicaciones`), usa WSL (Ubuntu). Ver docs/DESPLIEGUE.md.

Estructura

```
AI-Chatbot-Mizushima/
├── mcp_server.py              Servidor MCP: tools, recursos, prompts, rutas / y /salud
├── skills/                    Skills del servidor (se publican como skills://<nombre>)
├── agente_mcp.py              Mizushima: agente por terminal (cliente MCP)
├── agente_langchain.py        Base del agente: modelos, memoria, tools propias
├── agent-skills/
│   └── mizushima-mcp/SKILL.md Skill para agentes: cómo usar este MCP
├── verificar_despliegue.py    Revisa un despliegue contra los criterios
├── tests/test_servidor.py     Pruebas automáticas (APIs simuladas)
├── render.yaml                Despliegue en Render (Blueprint)
├── requirements-server.txt    Dependencias del servidor (Render)
├── requirements.txt           Todo: servidor + agente + pruebas
├── .env.example               Variables de entorno
├── .vscode/                   Tareas, depuración y mcp.json para VS Code
└── docs/
    ├── CODIGO.md              Arquitectura y explicación del código
    ├── AGENTE.md              Agente Mizushima: cómo funciona y comandos
    ├── SKILLS.md              Skills del servidor y skill para agentes
    ├── DESPLIEGUE.md          Local, ngrok, Render y WSL paso a paso
    ├── CLIENTES.md            Cómo conectarse desde cualquier cliente
    ├── REFERENCIA.md          Detalle de tools, recursos y prompts
    └── ENTREGA.md             Checklist y correo de entrega
```

Agregar cosas

- **Skill:** desde el agente con `/nueva-skill <nombre>`, desde cualquier cliente con la tool `crear_skill`, o creando `skills/<nombre>.md` en el código. Se publica sola como `skills://<nombre>`.
- **Tool:** una función con `@mcp.tool()` en `mcp_server.py`.
- **Prompt:** una función con `@mcp.prompt()` en `mcp_server.py`.

Documentación

- Código: arquitectura y funcionamiento
- Agente Mizushima
- Skills: del servidor y para agentes
- Despliegue: local, ngrok y Render
- Conectarse desde cualquier cliente
- Referencia de tools, recursos y prompts
- Entrega: checklist y correo

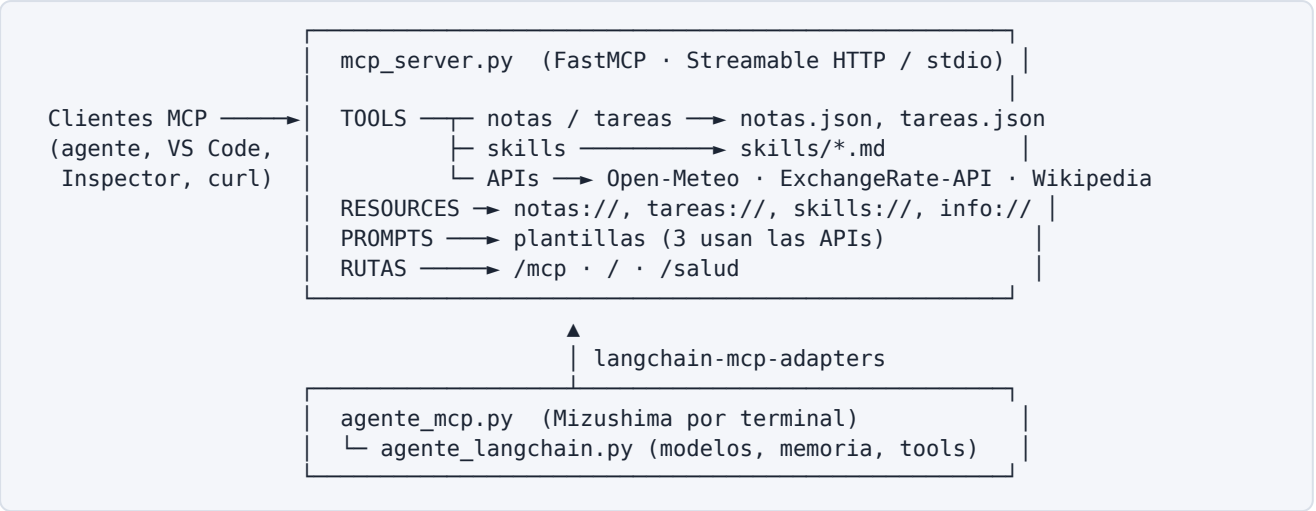
APIs externas y atribución

- Clima: Open-Meteo, licencia CC BY 4.0. Respaldo: wttr.in.
- Tasas de cambio: Rates By Exchange Rate API.
- Enciclopedia: Wikipedia, licencia CC BY-SA.

Ninguna requiere clave de API.

Documentación del código

Arquitectura



- **Servidor y agente están separados.** El servidor no usa ningún modelo de IA ni necesita claves. El agente es un cliente más del servidor, igual que VS Code o MCP Inspector.
- **El servidor no guarda sesión** (`stateless_http=True`) y **responde JSON** (`json_response=True`). Cada petición es independiente: funciona detrás de Render o ngrok y con clientes simples como `curl`.
- **Sin autenticación.** Para proteger lo básico, las skills base del proyecto no se pueden borrar ni reemplazar desde un cliente.

Archivos

Archivo	Líneas	Qué contiene
`mcp_server.py`	~530	El servidor MCP completo: configuración, tools, recursos, prompts y rutas HTTP
`agente_mcp.py`	~260	Mizushima: interfaz de terminal, conexión al servidor y comandos `/…`
`agente_langchain.py`	~120	Base del agente: modelos, memoria corta y larga, tools propias y prompt de sistema
`verificar_despliegue.py`	~90	Revisa un servidor desplegado contra los criterios de aceptación
`tests/test_servidor.py`	~240	16 pruebas automáticas (APIs simuladas y servidor HTTP real)
`skills/*.md`	—	Skills del servidor, publicadas como `skills://<nombre>`

Archivo	Líneas	Qué contiene
`agent-skills/mizushima-mcp/SKILL.md`	—	Skill para agentes que usan este servidor
`render.yaml`	—	Despliegue en Render (Blueprint)
`requirements-server.txt` / `requirements.txt`	—	Dependencias del servidor / de todo el proyecto

`mcp_server.py`, sección por sección

1. Configuración

Elemento	Qué hace
`HOST`, `PORT`, `ALLOWED_HOSTS`	Se leen de variables de entorno. En Render, `HOST=0.0.0.0` y Render pone el `PORT`
`seguridad (TransportSecuritySettings)`	Protección contra DNS rebinding. Activa solo en `127.0.0.1`; con ngrok se usa `--host-header=rewrite`
`mcp = FastMCP(...)`	Crea el servidor con nombre `mizushima`, instrucciones para los clientes, `stateless_http` y `json_response`
`_load` / `_save`	Leen y escriben JSON en UTF-8
`_get_json(url, params)`	Única función que sale a internet (httpx, 15 s de timeout, User-Agent propio). Las pruebas la reemplazan

2. Tools

Grupo	Funciones	Detalle
Notas	`guardar_nota`, `buscar_notas`, `listar_notas`, `borrar_nota`	Diccionario `{titulo: texto}` en `notas.json`
Tareas	`agregar_tarea`, `listar_tareas`, `completar_tarea`	Lista de `{id, texto, prioridad, hecha}` en `tareas.json`. Valida la prioridad
Skills	`listar_skills`, `crear_skill`, `borrar_skill`	`_nombre_skill()` normaliza el nombre (quita tildes, solo `a-z0-9_-`, máximo 50). Límite de 20 000 caracteres. `SKILLS_BASE` protege las skills del proyecto
Utilidad	`hora_actual`	Usa `zoneinfo` con zonas IANA
APIs	`clima`, `convertir_moneda`, `buscar_wikipedia`	Ver abajo

Manejo de las APIs:

- `_get_json_reintento()` reintenta **una vez**, tras 1,5 s, si el servicio responde 429 o 5xx o no contesta a tiempo.
- `_codigo_http(e)` convierte el error en texto legible (`HTTP 429`, `ConnectTimeout`) para el mensaje al usuario.
- Ninguna tool lanza excepciones al cliente. Ante un error devuelve un texto claro y el servidor sigue funcionando.

`clima`:

1. Revisa el caché (`_CACHE_CLIMA`, 10 minutos por ciudad y número de días).
2. Geocodifica la ciudad con Open-Meteo Geocoding.
3. Pide el pronóstico con Open-Meteo Forecast. `_formato_open_meteo()` arma el texto y traduce los códigos WMO al español con `CLIMA_WMO`.
4. Si Open-Meteo falla, `_clima_respaldo()` consulta **wttr.in** (formato `j1`, máximo 3 días).

`convertir_moneda`: consulta `open.er-api.com/v6/latest/{de}`. Una respuesta 404 significa que la moneda de origen no es válida. Revisa la de destino en `rates`. Muestra el monto, la tasa y la fecha de actualización, con la atribución que exige el servicio.

`buscar_wikipedia`: busca con `/w/rest.php/v1/search/page` y luego pide el resumen con `/api/rest_v1/page/summary/{key}`. Avisa si el resultado es una página de desambiguación. El idioma se valida con la expresión regular `[a-z]{2,3}`.

3. Recursos

Se definen con `@mcp.resource("esquema://ruta")`. Los que tienen `{variable}` son plantillas. `recurso_nota` decodifica `%20` con `unquote`. `recurso_skill` usa `Path(nombre).name` para que no se pueda salir de la carpeta `skills/`.

4. Prompts

Se definen con `@mcp.prompt()`. Cada uno devuelve el texto que el cliente le envía al modelo. Los que usan APIs empiezan su descripción con `[API …]`, y así los detectan las pruebas y el verificador. Cada prompt le indica al modelo qué skill leer antes de actuar.

5. Rutas HTTP y arranque

- `@mcp.custom_route("/")` devuelve un JSON con las tools, prompts y recursos disponibles.
- `/salud` es el health check de Render.
- `mcp.run(transport=MCP_TRANSPORT)` arranca el servidor en `stdio` (por defecto) o en `streamable-http`. En stdio no se usa `print()`, porque la salida estándar es el canal del protocolo.

Pruebas

```
python -m pip install -r requirements.txt
python -m pytest -v
```

Prueba	Qué verifica
`test_al_menos_cinco_tools_y_tres_de_internet`	Criterio: 5 o más tools, incluidas las 3 de APIs
`test_al_menos_tres_recursos`	Criterio: 3 o más recursos y las plantillas
`test_al_menos_dos_prompts_de_apis`	Criterio: 2 o más prompts de APIs
`test_apis_de_servicios_distintos`	Las 3 APIs son de servicios distintos
`test_clima`, `test_convertir_moneda`, `test_buscar_wikipedia`	Formato de cada respuesta
`test_clima_ciudad_inexistente`, `test_convertir_moneda_invalida`	Errores de entrada
`test_clima_usa_respaldo_si_open_meteo_falla`	Reintento y respaldo wttr.in ante un 429
`test_clima_usa_cache`	No repite la consulta en 10 minutos
`test_api_caida_no_rompe_el_servidor`	Si no hay red, responde un mensaje claro
`test_crear_leer_y_borrar_skill`, `test_skills_base_protegidas`	Gestión de skills
`test_nota_con_espacios_y_tareas`	Recursos con `%20` y tareas
`test_servidor_http_sin_autenticacion`	Levanta el servidor HTTP real y se conecta como cliente, sin token

Las APIs se simulan reemplazando `_get_json`, así que las pruebas no necesitan internet. La prueba real contra las APIs la hace `verificar_despliegue.py`.

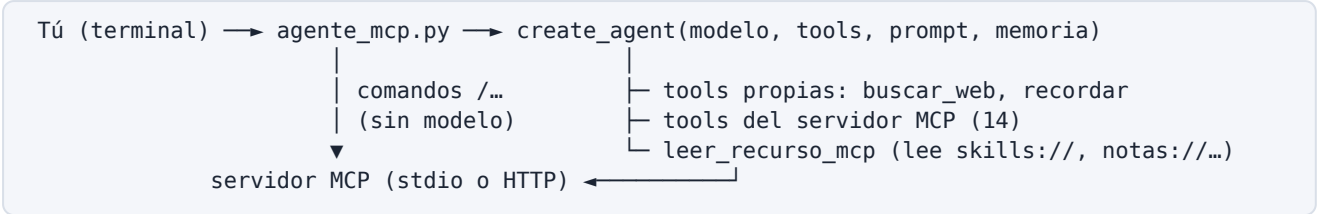
Extender

Quiero…	Cómo
Una tool nueva	Función con `@mcp.tool()` y su docstring. El docstring es lo que ve el modelo
Una tool que llame a una API	Usa `await _get_json_reintento(url, params)` y captura `httpx.HTTPError`
Un recurso	`@mcp.resource("esquema://ruta")`, con `{variable}` si es plantilla
Un prompt	`@mcp.prompt()`. Si usa una API, empieza la descripción con `[API Nombre]`
Una skill	Archivo `skills/<nombre>.md` o la tool `crear_skill` (ver SKILLS.md)

Documentación del agente (Mizushima)

Mizushima es un agente por terminal hecho con **LangChain** y **LangGraph**. Usa **OpenAI** o **Gemini** como modelo y se conecta al servidor MCP como cliente, ya sea en local, por ngrok o en Render. Así usa sus tools, recursos, prompts y skills.

Cómo funciona



1. Al arrancar, se conecta al servidor (`client.get_tools()`) y carga sus 14 tools como tools de LangChain.
2. Arma el agente con `create_agent(...)`: el modelo elegido, las tools (las suyas, las del servidor y `leer_recurso_mcp`), el prompt `SYSTEM_MCP` y la memoria corta (`InMemorySaver`).
3. Cada mensaje tuyo pasa por la memoria larga (`long_mem.search`). Si encuentra recuerdos relacionados, los agrega al mensaje.
4. El modelo decide qué tools usar. La terminal muestra cada una con 🔧 y luego la respuesta.
5. Los comandos que empiezan con `/` no pasan por el modelo: hablan directamente con el servidor.

Archivos

`agente_langchain.py`: la base

Elemento	Qué es
`MODELS`	Modelo de cada proveedor: `openai: <OPENAI_MODEL>` y `google_genai: <GOOGLE_MODEL>`, configurables en `.env`
`API_KEYS`, `PAQUETES`, `ALIAS`	Variable de clave y paquete de cada proveedor. `gemini` equivale a `google`
`checkpointer`	Memoria corta: el historial de la conversación mientras el programa corre
`LongMemory` / `long_mem`	Memoria larga en `memoria_larga.json`, con búsqueda por palabras en común
`buscar_web`	Tool propia: búsqueda en DuckDuckGo (`ddgs`)
`recordar`	Tool propia: guarda un dato del usuario en la memoria larga

Elemento	Qué es
`SYSTEM`	Prompt de sistema: se llama Mizushima, es conciso y responde en el idioma del usuario
`to_text`	Convierte el contenido de un mensaje (texto o bloques) en texto

También funciona solo, sin MCP: `python agente_langchain.py`.

`agente_mcp.py`: Mizushima con el servidor MCP

Elemento	Qué es
`SYSTEM_MCP`	`SYSTEM` más instrucciones para usar skills, `crear_skill` y las tools de APIs
Argumentos	`--url` (servidor remoto) y `--provider` (`openai` o `gemini`). También se leen de `MCP_URL` y `PROVIDER`
`conexion`	Sin `--url`, lanza `mcp_server.py` por **stdio**. Con `--url`, usa **Streamable HTTP**, con 90 s de espera para Render y `/mcp` agregado si falta
`leer_recurso_mcp`	Tool para que el modelo lea recursos (`skills://…`, `notas://…`)
`llamar_tool`, `leer`	Llaman al servidor directamente, sin modelo. Las usan los comandos `/`
`nueva_skill`	`/nueva-skill`: lee la skill de la terminal (hasta `FIN`) o de un archivo y la crea con `crear_skill`. Pregunta antes de reemplazar
`revisar_proveedor`	Antes de cambiar de modelo, revisa que el paquete esté instalado y que exista la clave
`_causa`	Saca el error real de los `ExceptionGroup` para mostrar mensajes claros (421, sin conexión…)
`ask`	Envía el mensaje al agente y muestra las tools usadas y la respuesta
`main`	Conecta, arma el agente y atiende los comandos

Instalación y uso

```
        python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
        python -m pip install -r requirements.txt
        cp .env.example .env                                  # pon OPENAI_API_KEY y/o
GOOGLE_API_KEY
```

```
        python agente_mcp.py                                          # servidor local
(stdio)
        python agente_mcp.py --url https://mizushima-mcp.onrender.com     # servidor en Render
        python agente_mcp.py --url https://xxxx.ngrok-free.app            # servidor por ngrok
        python agente_mcp.py --url https://mizushima-mcp.onrender.com --provider gemini
```

Comandos

Comando	Qué hace
(texto normal)	Habla con Mizushima
`/recursos`	Lista las tools, recursos, plantillas y prompts del servidor
`/skills`	Lista las skills del servidor
`/skill <nombre>`	Muestra una skill
`/nueva-skill <nombre>`	Crea una skill escribiéndola en la terminal. Termina con una línea `FIN`
`/nueva-skill <nombre> <archivo.md>`	Crea una skill desde un archivo
`/borrar-skill <nombre>`	Borra una skill creada por usuarios (pide confirmación)
`/prompt <nombre> clave=valor …`	Usa un prompt del servidor. Ej: `/prompt presupuesto_viaje destino=Madrid monto=3000000 moneda_destino=EUR`
`/provider`	Muestra el modelo actual
`/provider openai` · `/provider gemini`	Cambia de modelo **sin perder la conversación**
`/ayuda`, `/salir`	Ayuda y salir

Ejemplo de sesión

```
  ── AI-Chatbot-Mizushima ──
  Servidor MCP: https://mizushima-mcp.onrender.com/mcp
  Tools: 17  (14 del servidor MCP)
  Modelo: openai (…)

Tú: ¿cuánto son 100 dólares en pesos colombianos?
  🔧 leer_recurso_mcp({'uri': 'skills://consultas_externas'})
  🔧 convertir_moneda({'monto': 100, 'de': 'USD', 'a': 'COP'})

Mizushima: 100 USD son 323.641,77 COP (tasa del 9 de octubre de 2026). Fuente: ExchangeRate-
API.

Tú: /provider gemini
✔ Ahora usando google (…). La conversación continúa.
```

Configuración (`.env`)

Variable	Uso
`OPENAI_API_KEY` / `GOOGLE_API_KEY`	Claves de los modelos
`PROVIDER`	Con cuál arranca: `openai` o `google`
`OPENAI_MODEL` / `GOOGLE_MODEL`	Nombre del modelo. Cámbialo si el agente dice que el modelo no existe
`MCP_URL`	Servidor por defecto, igual que `--url`

Errores comunes

Mensaje	Solución
Falta OPENAI_API_KEY / GOOGLE_API_KEY	Pon la clave en `.env` y reinicia el agente
Falta instalar el paquete de google	`python -m pip install langchain-google-genai`
No me pude conectar al servidor MCP	Revisa la URL. En Render gratis, espera un minuto y reintenta
El servidor rechazó el dominio (421)	Inicia ngrok con `--host-header=rewrite`
El modelo no existe	Cambia `OPENAI_MODEL` o `GOOGLE_MODEL` en `.env`
`DLL load failed … Control de aplicaciones` (Windows)	Usa WSL (ver DESPLIEGUE.md)

Documentación de las skills

El proyecto tiene dos tipos de skill:

Tipo	Dónde está	Quién la usa	Para qué
Skills del servidor	`skills/*.md`	Cualquier cliente del servidor MCP, que las lee como recurso `skills://<nombre>`	Guías de buenas prácticas que el modelo lee antes de actuar
Skill para agentes	`agent-skills/mizushima-mcp/SKILL.md`	Agentes con soporte de skills (Claude Code, etc.)	Le enseña a un agente cuándo y cómo usar este servidor MCP

---

1. Skills del servidor

Son archivos Markdown con pasos numerados. El servidor publica cada archivo como recurso: `skills/redaccion.md` queda disponible en `skills://redaccion`. La lista completa está en `skills://indice` y en la tool `listar_skills`.

Skills incluidas

Skill	Recurso	Cuándo se usa	Prompts que la piden
Consultas a internet	`skills://consultas_externas`	Antes de usar `clima`, `convertir_moneda` o `buscar_wikipedia`: unidades, fecha de la tasa, ambigüedades (“pesos” de qué país), citar fuentes y no inventar datos	`reporte_clima`, `presupuesto_viaje`
Investigar en la web	`skills://investigacion_web`	Antes de investigar un tema: empezar con Wikipedia, contrastar, citar URLs, separar hechos de opiniones	`investigar_tema`
Organizar tareas	`skills://organizacion_de_tareas`	Antes de crear o planear tareas: verbo inicial, máximo 2 horas, prioridades y máximo 3 tareas de prioridad alta al día	`plan_del_dia`
Redacción clara	`skills://redaccion`	Antes de escribir o reescribir: conclusión primero, frases cortas, voz activa	`redactar_con_estilo`

Estas cuatro son **skills base**: ningún cliente las puede borrar ni reemplazar, aunque el servidor no tenga autenticación.

Cómo las usa el modelo

1. El prompt de sistema del agente y las instrucciones del servidor dicen: *"Antes de redactar, planear o consultar internet, lee la skill correspondiente."*
2. Los prompts del servidor lo piden explícitamente. Por ejemplo, `reporte_clima` empieza con *"Lee skills://consultas_externas"*.
3. El modelo lee la skill con `leer_recurso_mcp` (Mizushima) o con `resources/read` (otros clientes) y sigue sus pasos.

Crear una skill del servidor

Forma	Cómo	Dónde queda
Comando del agente	`/nueva-skill revision_de_codigo` y luego escribir los pasos y `FIN`	En el servidor al que esté conectado el agente (local o Render)
Desde un archivo	`/nueva-skill revision_de_codigo mi_skill.md`	Igual
Pidiéndoselo al agente	*"crea una skill para revisar código"*. Mizushima la redacta y usa `crear_skill`	Igual
Cualquier cliente MCP	Tool `crear_skill(nombre, contenido)`	Igual
Código	Crear `skills/<nombre>.md` y subirlo a GitHub	Permanente: Render la publica en el siguiente despliegue

Reglas: - **Nombre:** se normaliza a minúsculas sin tildes, con `_` o `-`. Por ejemplo, "Revisión de Código" queda como `revision_de_codigo`. - **Tamaño:** máximo 20 000 caracteres. - **Encabezado:** si no empieza con `#`, se agrega `# Skill: <nombre>`. - **Duración en Render:** las skills creadas con la tool viven mientras el servidor no se reinicie, porque el disco de Render es temporal. Para que sea permanente, agrégala al repositorio.

Plantilla:

```
# Skill: <título>

Cuando <situación>:
1. <primer paso concreto>.
2. <segundo paso>.
3. <cómo verificar o qué entregar>.
```

---

2. Skill para agentes: `mizushima-mcp`

Está en `agent-skills/mizushima-mcp/SKILL.md` y sigue el formato estándar de *Agent Skills*: una carpeta con un `SKILL.md` que empieza con un encabezado YAML.

```
        ---
        name: mizushima-mcp
        description: Usa el servidor Mizushima MCP para consultar clima (Open-Meteo), convertir
monedas (ExchangeRate-API), buscar en Wikipedia, y gestionar notas, tareas y skills. Úsala
cuando…
        ---
```

El agente lee la `description` para decidir **cuándo** usar la skill. El cuerpo le dice **cómo** hacerlo.

Qué contiene

Sección	Contenido
Conexión	Endpoint de Render, transporte, autenticación (ninguna), arranque en frío, configuración JSON y `mcp-remote` para clientes que solo usan stdio
Elegir la herramienta	Tabla “el usuario quiere… → usa…” con las 14 tools y sus argumentos
Recursos	Qué recursos hay y qué skill del servidor leer antes de cada tipo de tarea
Prompts listos	Los 9 prompts y cuáles usan APIs
Flujos recomendados	Viaje (moneda + clima), investigación (Wikipedia + nota), crear una skill
Reglas	Citar fuentes, no inventar datos, ambigüedades, espacio compartido, borrar solo si lo piden
Ejemplos	Pedidos reales y la llamada exacta que corresponde

Instalarla en un agente

- **Claude Code:** copia la carpeta a `.claude/skills/mizushima-mcp/` en tu proyecto o a `~/.claude/skills/mizushima-mcp/`.
- **Otros agentes con soporte de skills:** copia la carpeta `mizushima-mcp` a su carpeta de skills.
- **Agentes sin soporte de skills:** pega el contenido de `SKILL.md`, sin el encabezado YAML, en el prompt de sistema.

Además, el agente debe tener configurado el servidor MCP. Ver CLIENTES.md.

Referencia del servidor MCP

Tools (14)

Consumen APIs de internet

Tool	Argumentos	Servicio	Devuelve
`clima`	`ciudad: str`, `dias: int = 3` (1–7)	**Open-Meteo**: geocodificación y pronóstico	Clima actual (temperatura, sensación, humedad, viento) y pronóstico diario con probabilidad de lluvia
`convertir_moneda`	`monto: float`, `de: str = "USD"`, `a: str = "COP"`	**ExchangeRate-API** (`open.er-api.com`)	Monto convertido, tasa y fecha de actualización
`buscar_wikipedia`	`tema: str`, `idioma: str = "es"`	**Wikipedia REST**: búsqueda y resumen	Título, descripción, resumen y enlace

Las tres son servicios distintos, gratuitos y sin clave. Si un servicio está saturado (HTTP 429 o 5xx), la tool reintenta una vez. Si aun así no responde, devuelve un mensaje claro con el código del error (“No pude consultar… (HTTP 429)”) y el servidor sigue funcionando.

`clima` tiene además un **respaldo**: si Open-Meteo falla (pasa a veces en Render, porque muchos servicios comparten sus IP y Open-Meteo las limita), consulta **wttr.in**, que tampoco pide clave. Los resultados del clima se guardan 10 minutos para no repetir consultas.

Notas

Tool	Argumentos	Qué hace
`guardar_nota`	`titulo`, `texto`	Guarda o sobrescribe una nota
`buscar_notas`	`consulta`	Busca en títulos y textos
`listar_notas`	—	Títulos de todas las notas
`borrar_nota`	`titulo`	Borra una nota

Tareas

Tool	Argumentos	Qué hace
`agregar_tarea`	`texto`, `prioridad = "media"` (alta, media o baja)	Crea una tarea
`listar_tareas`	`solo_pendientes = True`	Lista tareas
`completar_tarea`	`id`	Marca una tarea como hecha

Skills

Tool	Argumentos	Qué hace
`listar_skills`	—	Nombre y título de cada skill
`crear_skill`	`nombre`, `contenido`, `sobrescribir = False`	Crea `skills/<nombre>.md`. El nombre se normaliza: “Revisión de Código” → `revision_de_codigo`. Máximo 20 000 caracteres
`borrar_skill`	`nombre`	Borra una skill creada por usuarios

Las skills base (`redaccion` , `organizacion_de_tareas` , `investigacion_web` , `consultas_externas`) no se pueden borrar ni reemplazar desde un cliente. Así se protegen, aunque el servidor no tenga autenticación.

Utilidades

Tool	Argumentos	Qué hace
`hora_actual`	`zona = "America/Bogota"`	Fecha y hora en una zona IANA

Recursos (6 fijos y 2 plantillas)

URI	Contenido
`info://servidor`	Versión, APIs externas que usa y cómo agregar skills
`notas://lista`	Títulos de las notas
`notas://todas`	Todas las notas con su texto
`notas://{titulo}`	Una nota. Los espacios van como `%20`
`tareas://pendientes`	Tareas sin completar
`tareas://todas`	Todas las tareas
`skills://indice`	Nombres de las skills
`skills://{nombre}`	Texto de una skill

Prompts (9)

Prompt	Argumentos	APIs	Qué pide al modelo
`reporte_clima`	`ciudad`	Open-Meteo	Reporte del clima con 3 recomendaciones
`presupuesto_viaje`	`destino`, `monto`, `moneda_origen = "COP"`, `moneda_destino = "USD"`	ExchangeRate-API + Open-Meteo	Presupuesto convertido, clima de 5 días, qué empacar y gasto diario sugerido

Prompt	Argumentos	APIs	Qué pide al modelo
`investigar_tema`	`tema`	Wikipedia	Resumen en 5 puntos guardado como nota
`resumir_notas`	—	—	Resumen de las notas en 3 líneas
`plan_del_dia`	—	—	Plan por prioridades con horarios
`revisar_tareas`	—	—	Estado de las tareas y qué hacer primero
`notas_a_tareas`	`titulo`	—	Convierte una nota en tareas
`explicar_tema`	`tema`, `nivel = "básico"`	—	Explicación con ejemplo
`redactar_con_estilo`	`texto`	—	Reescribe con la skill de redacción

Los prompts que usan APIs empiezan su descripción con `[API …]`.

Skills del servidor

Skill	Para qué
`consultas_externas`	Cómo usar clima, monedas y Wikipedia: unidades, fechas, fuentes y errores
`investigacion_web`	Cómo investigar y citar fuentes
`organizacion_de_tareas`	Cómo escribir y priorizar tareas
`redaccion`	Cómo redactar textos claros

Rutas HTTP

Ruta	Método	Uso
`/mcp`	POST	Endpoint MCP (Streamable HTTP, respuestas JSON, sin sesión)
`/`	GET	Información del servidor en JSON: tools, prompts, recursos y endpoint
`/salud`	GET	`{"estado": "ok"}`, para el health check de Render

Conectarse desde cualquier cliente

El servidor usa el transporte estándar **Streamable HTTP** y no pide autenticación. Cualquier cliente MCP se conecta con la URL del endpoint:

```
https://mizushima-mcp.onrender.com/mcp      (Render: reemplaza por tu URL)
https://xxxx.ngrok-free.app/mcp             (ngrok)
http://127.0.0.1:8000/mcp                   (local)
```

> En Render gratis, la primera petición después de 15 minutos sin uso tarda unos 50 segundos.

1. Mizushima (agente por terminal)

```
python agente_mcp.py --url https://mizushima-mcp.onrender.com
python agente_mcp.py --url https://mizushima-mcp.onrender.com --provider gemini
```

No hace falta poner `/mcp` al final: el agente lo agrega solo.

Comando	Qué hace
`/recursos`	Lista tools, recursos y prompts
`/skills`, `/skill <nombre>`	Lista o muestra skills
`/nueva-skill <nombre>`	Crea una skill escribiéndola en la terminal. Termina con `FIN`
`/nueva-skill <nombre> <archivo.md>`	Crea una skill desde un archivo
`/borrar-skill <nombre>`	Borra una skill creada por usuarios
`/prompt <nombre> clave=valor`	Usa un prompt. Ej: `/prompt reporte_clima ciudad=Bogotá`
`/provider [openai|gemini]`	Muestra o cambia el modelo sin perder la conversación
`/ayuda`, `/salir`	Ayuda y salir

También puedes hablarle normal: *“¿cuánto son 100 dólares en pesos colombianos?”*.

2. MCP Inspector (navegador, sin instalar nada)

Requiere Node.js:

```
npx @modelcontextprotocol/inspector
```

En la página que se abre, elige **Transport Type: Streamable HTTP**, pega la URL con `/mcp` y haz clic en **Connect**. Ahí puedes probar cada tool, recurso y prompt.

3. VS Code (Copilot Chat en modo Agent)

El proyecto trae `.vscode/mcp.json` con dos entradas:

- `mizushima-local`: lanza el servidor en tu PC.
- `mizushima-remoto`: pide la URL de Render o ngrok, sin `/mcp`.

Pulsa **Start** sobre la que quieras y abre el chat en modo **Agent**. En otro proyecto puedes agregarlo así:

```
        { "servers": { "mizushima": { "type": "http", "url": "https://mizushima-
mcp.onrender.com/mcp" } } }
```

4. Claude Desktop y otros clientes que solo usan stdio

Usa el puente `mcp-remote`, que requiere Node.js. En el archivo de configuración del cliente:

```
        {
          "mcpServers": {
            "mizushima": {
              "command": "npx",
              "args": ["-y", "mcp-remote", "https://mizushima-mcp.onrender.com/mcp"]
            }
          }
        }
```

Los clientes que aceptan servidores remotos por URL (conectores personalizados) solo necesitan `https://mizushima-mcp.onrender.com/mcp`.

5. curl (JSON-RPC directo)

El servidor responde JSON y no guarda sesión, así que cada petición funciona sola:

```
        URL=https://mizushima-mcp.onrender.com/mcp

        # Listar tools
        curl -s $URL -H "Content-Type: application/json" -H "Accept: application/json,
text/event-stream" \
          -d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'

        # Clima de Medellín
        curl -s $URL -H "Content-Type: application/json" -H "Accept: application/json,
text/event-stream" \
          -d '{"jsonrpc":"2.0","id":2,"method":"tools/call","params":
{"name":"clima","arguments":{"ciudad":"Medellín","dias":2}}}'

        # Convertir 100 USD a COP
        curl -s $URL -H "Content-Type: application/json" -H "Accept: application/json,
text/event-stream" \
          -d '{"jsonrpc":"2.0","id":3,"method":"tools/call","params":
{"name":"convertir_moneda","arguments":{"monto":100,"de":"USD","a":"COP"}}}'

        # Leer un recurso
        curl -s $URL -H "Content-Type: application/json" -H "Accept: application/json,
text/event-stream" \
          -d '{"jsonrpc":"2.0","id":4,"method":"resources/read","params":
{"uri":"skills://consultas_externas"}}'
```

6. Desde Python (SDK oficial de MCP)

```
        import asyncio
        from mcp import ClientSession
        from mcp.client.streamable_http import streamable_http_client

        async def main():
            async with streamable_http_client("https://mizushima-mcp.onrender.com/mcp") as (r,
w, _):
                async with ClientSession(r, w) as s:
                    await s.initialize()
                    res = await s.call_tool("convertir_moneda", {"monto": 50, "de": "EUR", "a":
"COP"})
                    print(res.content[0].text)

        asyncio.run(main())
```

7. Agentes con skills

Copia la carpeta `agent-skills/mizushima-mcp` a la carpeta de skills de tu agente, por ejemplo `.claude/skills/` en Claude Code. El agente aprende cuándo y cómo usar cada tool, recurso y prompt de este servidor.

Despliegue

El servidor se puede desplegar de tres formas. Ninguna pide autenticación.

Forma	URL	Disponible	Para qué
Local (stdio)	—	Mientras lo uses	Desarrollar y probar en tu PC
Local + ngrok	`https://xxxx.ngrok-free.app/mcp`	Mientras tu PC y ngrok estén encendidos	Mostrarlo desde tu PC a otros equipos
Remoto en Render	`https://mizushima-mcp.onrender.com/mcp`	Siempre (se duerme si no se usa)	**La entrega**: cualquiera lo usa cuando quiera

---

1. Local (stdio)

```
python -m pip install -r requirements.txt
python agente_mcp.py
```

El agente lanza `mcp_server.py` como subproceso. En VS Code: abre `.vscode/mcp.json` y pulsa **Start** sobre `mizushima-local`.

2. Local por HTTP + ngrok

Servidor HTTP

```
MCP_TRANSPORT=streamable-http python mcp_server.py
# Windows PowerShell:
$env:MCP_TRANSPORT="streamable-http"; python mcp_server.py
```

Queda en `http://127.0.0.1:8000/mcp`. Al abrir `http://127.0.0.1:8000` en el navegador ves la información del servidor.

Instalar ngrok (una sola vez)

En Ubuntu / WSL:

```
curl -sSL https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee
/etc/apt/trusted.gpg.d/ngrok.asc >/dev/null
echo "deb https://ngrok-agent.s3.amazonaws.com bookworm main" | sudo tee
/etc/apt/sources.list.d/ngrok.list
sudo apt update && sudo apt install -y ngrok
ngrok config add-authtoken TU_AUTHTOKEN     # está en dashboard.ngrok.com
```

En Windows: `winget install ngrok.ngrok` y luego el mismo `ngrok config add-authtoken`.

Exponer el servidor

Con el servidor HTTP corriendo, en otra terminal:

```
ngrok http 8000 --host-header=rewrite
```

Copia la URL `https://xxxx.ngrok-free.app` de la línea *Forwarding*. El endpoint es esa URL más `/mcp`. En VS Code, la tarea **Despliegue local: servidor + ngrok** hace los dos pasos.

> **¿Por qué `--host-header=rewrite`?** El SDK de MCP rechaza con `421 Invalid Host header` las peticiones cuyo Host no es local, como protección contra DNS rebinding. Si prefieres no usar esa opción, pon tu dominio de ngrok en `ALLOWED_HOSTS`.

Limitaciones: el servidor solo funciona mientras tu PC, el servidor y ngrok estén encendidos. Con el plan gratis, la URL cambia cada vez que reinicias ngrok, a menos que actives el dominio fijo gratis en *Domains* y lo uses con `--url=tu-dominio.ngrok-free.app`.

3. Remoto en Render (recomendado para la entrega)

Render ejecuta el servidor en la nube, sin tu PC. El archivo `render.yaml` ya trae toda la configuración.

Paso 1: subir el código a GitHub

Opción A, desde el navegador (sin git):

1. Entra a github.com/new, ponle de nombre `AI-Chatbot-Mizushima`, márcalo **Public** y crea el repositorio.
2. Haz clic en **uploading an existing file**.
3. Arrastra **todo el contenido** de la carpeta del proyecto, incluidas `skills/`, `agent-skills/`, `docs/`, `tests/` y `.vscode/`. **No subas `.env`**: tiene tus claves.
4. Haz clic en **Commit changes**.

Opción B, con git (desde Ubuntu / WSL):

```
cd ~/mizushima
git init && git add . && git commit -m "Mizushima MCP"
git branch -M main
git remote add origin https://github.com/heroleni/AI-Chatbot-Mizushima.git
git push -u origin main
```

El archivo `.gitignore` ya evita subir `.env`, `.venv` y los datos locales. Para el `push`, GitHub pide un token personal como contraseña. También puedes usar `gh auth login`.

Paso 2: crear el servicio en Render

1. Entra a dashboard.render.com e inicia sesión **con tu cuenta de GitHub**.
2. Haz clic en **New → Blueprint**.
3. Conecta tu repositorio `AI-Chatbot-Mizushima`. Render detecta `render.yaml` y muestra el servicio `mizushima-mcp` (plan *Free*).
4. Haz clic en **Deploy Blueprint**. La primera vez tarda unos 2 o 3 minutos.

5. Abre el servicio y copia su URL, algo como `https://mizushima-mcp.onrender.com`. Si el nombre ya estaba ocupado, Render le agrega un sufijo.

Si no te aparece *Blueprint*, usa **New → Web Service** con estos valores:

Campo	Valor
Runtime	Python 3
Build Command	`pip install -r requirements-server.txt`
Start Command	`python mcp_server.py`
Instance Type	Free
Environment Variables	`MCP_TRANSPORT=streamable-http`, `HOST=0.0.0.0`, `PYTHON_VERSION=3.12.7`
Health Check Path	`/salud`

Paso 3: verificar

1. Abre la URL en el navegador. Debe mostrar un JSON con `"autenticacion": "ninguna"` y la lista de tools.
2. Ejecuta el verificador:

   ```
   python verificar_despliegue.py https://mizushima-mcp.onrender.com
   ```

   Todos los puntos deben salir con ✔. El verificador llama de verdad a las 3 APIs de internet.

Cosas que debes saber de Render gratis

- **Se duerme:** tras 15 minutos sin uso, el servidor se apaga. La siguiente petición lo despierta en unos 50 segundos. El agente espera hasta 90 segundos. Antes de una demo, abre la URL en el navegador para despertarlo.
- **Disco temporal:** las notas, tareas y skills creadas mientras corre se borran cuando Render reinicia o vuelve a desplegar. Las skills que están en `skills/` del repositorio se mantienen. Para que una skill sea permanente, agrégala al repositorio: con `autoDeploy`, Render la publica sola.
- **Actualizar:** cada cambio que subas a GitHub se despliega automáticamente.

4. Red local (sin ngrok)

```
MCP_TRANSPORT=streamable-http HOST=0.0.0.0 python mcp_server.py
```

Los otros equipos de la misma red usan `http://IP-DE-TU-PC:8000/mcp`. Si el servidor corre dentro de WSL, los otros equipos no lo ven directamente, porque WSL tiene su propia red. En ese caso usa ngrok.

Windows 11 y el Control de aplicaciones

Windows 11 puede bloquear piezas compiladas de algunas librerías con el error `DLL load failed … Una directiva de Control de aplicaciones bloqueó este archivo`. Las opciones son:

1. **Usar WSL (recomendado):**

```
sudo apt update && sudo apt install -y python3-venv python3-pip
cp -r "/mnt/d/ruta/al/proyecto" ~/mizushima && cd ~/mizushima
rm -rf .venv && python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
```

2. Usar siempre `python -m pip` en vez de `pip`. Así se evita el bloqueo de `pip.exe`.
3. Desactivar el *Control inteligente de aplicaciones* en Seguridad de Windows. En muchas versiones, una vez apagado no se puede volver a encender sin reinstalar Windows.

Variables de entorno

Variable	Dónde	Valor por defecto	Uso
`MCP_TRANSPORT`	servidor	`stdio`	`streamable-http` para servir por HTTP
`HOST`	servidor	`127.0.0.1`	`0.0.0.0` en Render o en red local
`PORT`	servidor	`8000`	Render lo asigna solo
`ALLOWED_HOSTS`	servidor	—	Dominios extra permitidos (ngrok sin `--host-header=rewrite`)
`MCP_URL`	agente	—	Servidor remoto (igual que `--url`)
`OPENAI_API_KEY` / `GOOGLE_API_KEY`	agente	—	Claves de los modelos
`PROVIDER`	agente	`openai`	`openai` o `google`
`OPENAI_MODEL` / `GOOGLE_MODEL`	agente	ver `.env.example`	Nombre del modelo

Anexo A. Skill para agentes (SKILL.md)

```
---
name: mizushima-mcp
description: Usa el servidor Mizushima MCP para consultar clima (Open-Meteo), convertir monedas (ExchangeRate-API), buscar en Wikipedia, y gestionar notas, tareas y skills. Úsala cuando el usuario pida el clima de una ciudad, convertir dinero entre monedas, información enciclopédica de un tema, planear su día o guardar notas y tareas en Mizushima.
---

# Mizushima MCP

Servidor MCP público y sin autenticación. Combina tres APIs de internet con un espacio de notas, tareas y guías (skills) compartido entre todos los clientes.

## Conexión

- **Endpoint:** `https://mizushima-mcp.onrender.com/mcp` (reemplázalo por la URL del despliegue que te dieron)
- **Transporte:** Streamable HTTP. Responde JSON y no guarda sesión.
- **Autenticación:** ninguna.
- **Primera llamada lenta:** en el plan gratis de Render el servidor se duerme tras 15 minutos sin uso. Si la primera conexión falla por tiempo, espera unos 50 segundos y reintenta una vez.

Configuración típica de un cliente MCP:

```json
{ "mcpServers": { "mizushima": { "type": "http", "url": "https://mizushima-mcp.onrender.com/mcp" } } }
```

Si el cliente solo soporta stdio: `npx -y mcp-remote https://mizushima-mcp.onrender.com/mcp`.

Elegir la herramienta

El usuario quiere…	Usa	Notas
Clima actual o pronóstico	`clima(ciudad, dias=3)`	`dias` de 1 a 7. Nombre de ciudad simple: "Medellín", "Madrid"
Convertir dinero	`convertir_moneda(monto, de, a)`	Códigos ISO 4217: USD, COP, EUR, MXN… Tasa diaria
Saber qué es algo	`buscar_wikipedia(tema, idioma="es")`	Devuelve resumen y enlace
Hora en una zona	`hora_actual(zona)`	Zona IANA: America/Bogota
Guardar o consultar notas	`guardar_nota`, `buscar_notas`, `listar_notas`, `borrar_nota`	
Gestionar tareas	`agregar_tarea(texto, prioridad)`, `listar_tareas`, `completar_tarea(id)`	prioridad: alta, media o baja
Ver o crear guías	`listar_skills`, `crear_skill(nombre, contenido)`, `borrar_skill`	Las skills base no se pueden borrar ni reemplazar

Recursos que puedes leer

- `info://servidor`: qué APIs usa el servidor y cómo agregar skills
- `skills://indice` y `skills://{nombre}`: guías de buenas prácticas. **Léelas antes de actuar:**
  - `consultas_externas` antes de usar clima, monedas o Wikipedia
  - `investigacion_web` antes de investigar un tema
  - `organizacion_de_tareas` antes de crear o planear tareas
  - `redaccion` antes de escribir o reescribir textos
- `notas://todas`, `notas://lista`, `notas://{titulo}` (espacios como `%20`)
```

```
        - `tareas://pendientes`, `tareas://todas`

        ## Prompts listos

        | Prompt | APIs | Para qué |
        |---|---|---|
        | `reporte_clima(ciudad)` | Open-Meteo | Reporte del clima con recomendaciones |
        | `presupuesto_viaje(destino, monto, moneda_origen, moneda_destino)` | ExchangeRate-API
+ Open-Meteo | Presupuesto convertido y clima del destino |
        | `investigar_tema(tema)` | Wikipedia | Resumen en 5 puntos guardado como nota |
        | `plan_del_dia`, `revisar_tareas`, `resumir_notas`, `notas_a_tareas(titulo)` | — |
Trabajo con notas y tareas |
        | `explicar_tema(tema, nivel)`, `redactar_con_estilo(texto)` | — | Explicar y redactar
|

        ## Flujos recomendados

        **Viaje:** `convertir_moneda` del presupuesto, luego `clima` del destino con `dias=5`.
Al final resume el monto convertido, la fecha de la tasa, qué empacar y las fuentes.

        **Investigación:** lee `skills://investigacion_web`, usa `buscar_wikipedia`, resume con
enlace y, si el usuario quiere conservarlo, guárdalo con `guardar_nota` y el título
`Investigación: <tema>`.

        **Crear una skill:** escríbela como pasos numerados en Markdown, empezando con `#
Skill: <título>`. Usa `crear_skill` con un nombre corto en minúsculas, por ejemplo
`revision_de_codigo`. Usa `sobrescribir=True` solo si el usuario lo pide.

        ## Reglas

        1. **Cita la fuente** de los datos de internet: Open-Meteo, ExchangeRate-API o
Wikipedia, con enlace cuando exista.
        2. **No inventes datos.** Si una tool devuelve "No pude consultar…" o "No encontré…",
dilo y sugiere cómo corregir la consulta.
        3. **Ambigüedades:** si el usuario dice "pesos" sin decir de qué país, pregunta. Si
dice "dólares", asume USD y aclara.
        4. **Espacio compartido:** las notas, tareas y skills las ven todos los clientes. No
guardes datos personales sensibles.
        5. **Borrado:** borra notas, tareas o skills solo si el usuario lo pidió
explícitamente.

        ## Ejemplos

        - "¿Cómo está el clima en Cartagena esta semana?" → `clima(ciudad="Cartagena", dias=7)`
        - "¿Cuánto son 250 dólares en pesos colombianos?" → `convertir_moneda(monto=250,
de="USD", a="COP")`
        - "¿Qué es el Model Context Protocol?" → `buscar_wikipedia(tema="Model Context
Protocol")`. Si no hay resultado en español, reintenta con `idioma="en"`.
        - "Prepárame el viaje a Madrid con 3 millones de pesos" → prompt
`presupuesto_viaje(destino="Madrid", monto="3000000", moneda_origen="COP",
moneda_destino="EUR")`
```

Anexo B. Skills del servidor

skills://consultas_externas

```
        # Skill: consultas a internet (clima, monedas, Wikipedia)

        Cuando uses clima, convertir_moneda o buscar_wikipedia:
        1. Pide solo lo necesario: una ciudad concreta, códigos de moneda ISO (USD, COP, EUR) o
un tema específico.
        2. Si el usuario dice "pesos", "dólares" o "euros", traduce a COP, USD o EUR; si es
ambiguo (pesos de qué país), pregunta.
        3. Muestra las cifras con sus unidades: °C, %, km/h, y el código de moneda junto al
monto.
        4. En conversiones, indica siempre la fecha de la tasa: cambia todos los días.
        5. Cita la fuente al final: Open-Meteo, ExchangeRate-API o Wikipedia (con el enlace).
        6. Si la tool devuelve un error, explícalo en una línea y sugiere cómo corregir la
consulta; no inventes datos.
        7. Separa los datos de la API de tus recomendaciones u opiniones.
```

skills://investigacion_web

```
        # Skill: investigar en la web

        1. Empieza con buscar_wikipedia para tener una base confiable; usa consultas cortas y
específicas.
        2. Si el tema es reciente, aclara que Wikipedia puede no estar al día e incluye el año
en la búsqueda.
        3. Contrasta al menos 2 fuentes antes de afirmar algo importante, cuando tengas otra
fuente disponible.
        4. Prefiere fuentes primarias (sitios oficiales, documentación, papers) sobre blogs y
foros.
        5. Indica siempre la URL de cada dato.
        6. Si las fuentes se contradicen, dilo explícitamente en lugar de elegir una.
        7. Separa claramente hechos confirmados de opiniones o rumores.
```

skills://organizacion_de_tareas

```
        # Skill: organizar tareas

        1. Una tarea debe empezar con un verbo y poder terminarse en menos de 2 horas. Si es
más grande, divídela.
        2. Prioridad alta: tiene fecha límite cercana o bloquea a otros. Media: importante sin
urgencia. Baja: opcional.
        3. Máximo 3 tareas de prioridad alta por día.
        4. Al planear el día, pon primero lo de prioridad alta y deja un margen del 20% para
imprevistos.
        5. Al terminar el día, revisa lo pendiente y reprograma, no acumules.
```

skills://redaccion

```
# Skill: redacción clara

Cuando escribas o reescribas un texto:
1. Empieza por la conclusión o lo más importante.
2. Frases cortas: máximo 20 palabras.
3. Voz activa, sin rodeos ni relleno.
4. Usa listas solo si hay 3 o más elementos paralelos.
5. Termina con la acción siguiente, si la hay.
6. Mantén el idioma y el tono del texto original.
```